utad UNIVERSIDADE DE TRÁS-OS-MONTES E ALTO DOURO

BetStrike

Integração REST, Base de Dados, Middleware e Streaming de Eventos

Filipa Monteiro 78231
João Esteves 78411
Mestrado em Engenharia Informática, Grupo D

Unidade Curricular:	Integração de Sistemas
Docentes:	João Paulo Pereira de Sousa Hugo Alexandre Paredes Guedes da Silva

Vila Real, junho de 2026

Índice

1. Introdução 4
2. Arquitetura da Solução 5
3. Parte 1: Integração REST e Base de Dados 6
   3.1. Decisões de Design 6
   3.2. GeradorDados: Simulação da FPF 6
   3.3. Gestão de Estado dos Jogos 8
   3.4. Integração ApiApostas e ApiPagamentos 8
   3.5. Frontend Angular 8
      Funcionalidades implementadas 9
      Análise da relação com a API 11
   3.6. Containerização 11
4. Parte 2: Motivação para o Middleware 12
5. Apache Kafka: Streaming de Eventos 13
   5.1. Fundamentos e Adequação ao Problema 13
   5.2. Justificação da Escolha 13
   5.3. Implementação e Decisões 13
   5.4. Dois Tópicos, Dois Papéis 14
   5.5. Análise do Fluxo Kafka 14
6. RabbitMQ: Filas de Trabalho 16
   6.1. Fundamentos e Adequação ao Problema 16
   6.2. Justificação da Escolha 16
   6.3. Configuração e Estratégia de Retry 16
   6.4. BetResolutionWorker: Fluxo Completo 17
   6.5. Análise do Fluxo RabbitMQ 17
7. Idempotência e Resiliência 19
   7.1. O Problema da Deduplicação 19
   7.2. PendingBetsSweeper: Camada de Segurança 19
   7.3. Comportamento sob Falhas 19
8. Analytics, Dashboards e Alertas 21
   8.1. Abordagem Arquitetural 21

8.2. Métricas e Alertas 21
8.3. Análise da Abordagem 22
9. Testes e Resultados 23
9.1. Testes de Integração 23
9.2. Testes de Stress 23
9.3. Testes de Resiliência e Idempotência 23
10. Análise Crítica e Limitações 25
10.1. O que Funciona Bem 25
10.2. Limitações e Melhorias Futuras 25
11. Conclusão 26
12. Referências Bibliográficas 27
Anexo 1 - Código Fonte 28

1. Introdução

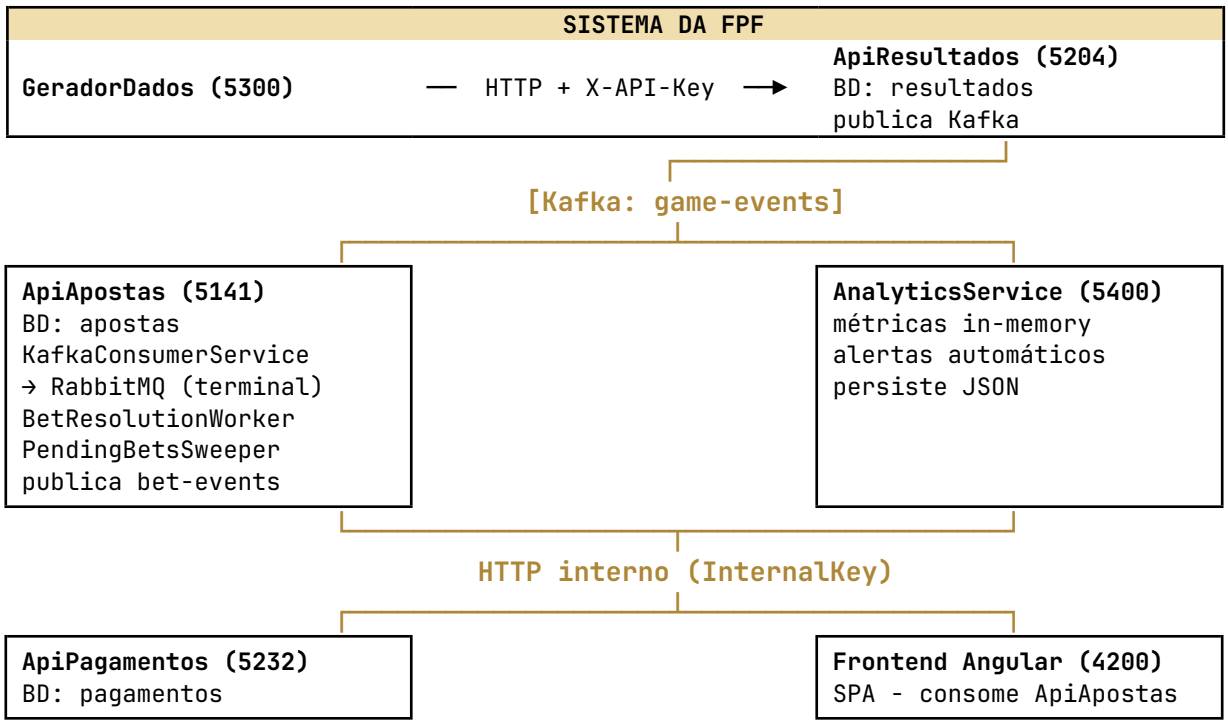
A BetStrike é uma empresa fictícia portuguesa de apostas desportivas online com mais de 200 000 utilizadores ativos mensais. Com o crescimento da plataforma surgiram três problemas estruturais: resultados de jogos inseridos manualmente, ausência de comunicação automática entre os sistemas de apostas e de pagamentos, e resolução de apostas com latências de 15 a 30 minutos, incompatíveis com as expectativas de uma plataforma live.

Este trabalho implementa a modernização da arquitetura em duas fases. A Parte 1 estabelece a integração REST entre os sistemas, com toda a lógica de negócio centralizada em Stored Procedures SQL Server. A Parte 2 introduz middleware de mensagens (Apache Kafka e RabbitMQ) para desacoplar os sistemas, reduzir latências e suportar processamento assíncrono, complementado por um microserviço de analytics em tempo real. Toda a infraestrutura é executada em ambiente containerizado com Docker Compose.

O conjunto das duas partes resulta num sistema distribuído completo que cobre o ciclo de vida de uma aposta desportiva: desde a publicação do calendário de jogos pela Federação Portuguesa de Futebol, passando pela simulação dos resultados em tempo real, até ao crédito automático de prémios e à monitorização operacional da plataforma.

2. Arquitetura da Solução

O sistema é composto por quatro serviços de backend, uma aplicação geradora de dados, um microserviço de analytics e um frontend Angular. A comunicação usa dois paradigmas: HTTP/REST para operações síncronas e transacionais, e mensagens assíncronas (Kafka e RabbitMQ) para propagação de eventos e processamento diferido. As três bases de dados SQL Server são logicamente separadas, com utilizadores e permissões restritos por serviço.



A separação entre a base de dados da FPF (betstrike-resultados) e a base de dados da BetStrike (betstrike-apostas) é uma decisão arquitetural deliberada. Os dois sistemas pertencem a organizações diferentes e não devem partilhar schema. A sincronização de dados entre eles é feita por integração, nunca por acesso direto à base de dados do outro sistema. Na Parte 1 esse canal é polling HTTP; na Parte 2 é consumo do stream Kafka.

3. Parte 1: Integração REST e Base de Dados

3.1. Decisões de Design

A Parte 1 resolve o problema mais imediato: eliminar a inserção manual de resultados e ligar automaticamente os sistemas da FPF e da BetStrike. A abordagem escolhida foi REST sobre HTTP, o estilo arquitetural mais adequado para integração entre sistemas heterogéneos sem partilha de infraestrutura. É stateless, usa verbos HTTP com semântica bem definida, e é naturalmente interoperável com qualquer stack tecnológico.

Na arquitetura definida para a Parte 1, o acesso à base de dados é feito exclusivamente através de Stored Procedures SQL Server. Esta abordagem traz vantagens concretas num sistema financeiro: a lógica de validação fica centralizada na camada de dados, o que significa que não pode ser bypassada por outro cliente que aceda à mesma base de dados; as SPs executam dentro de transações atómicas, eliminando race conditions como dois utilizadores a apostar com o mesmo saldo insuficiente em simultâneo; e as regras de negócio ficam num único local auditável, o que facilita a revisão e o controlo de alterações.

A contrapartida desta abordagem é o acoplamento entre a API e o esquema de base de dados: uma alteração numa SP pode quebrar silenciosamente a API se não houver testes de integração suficientes. É um tradeoff que faz sentido num contexto de sistema financeiro, onde a integridade transacional pesa mais do que a flexibilidade de schema.

3.2. GeradorDados: Simulação da FPF

O GeradorDados opera em duas fases sequenciais. A Fase 1 publica antecipadamente o calendário completo da jornada: gera 9 emparelhamentos para as 18 equipas da Primeira Liga, garantindo que cada equipa joga exatamente uma vez em casa e uma vez fora ao longo de duas jornadas consecutivas. Cada jogo recebe um código no formato FUT-AAAA-JJNN. A operação é idempotente: consulta os jogos existentes antes de publicar e aborta se a jornada já estiver registada, o que evita duplicados mesmo que o GeradorDados seja reiniciado durante a execução.

A Fase 2 simula os 9 jogos em paralelo como tarefas assíncronas independentes. A cada 10 segundos (10 minutos fictícios), o estado ou o marcador é atualizado via PATCH na ApiResultados. Golos são gerados aleatoriamente com média de 2 a 3 por jogo, incrementando progressivamente. O HttpClientService usa uma Polly retry policy com backoff exponencial (3 tentativas, delays de 1s, 2s e 4s) para absorver falhas transitórias. Esta resiliência no cliente é relevante porque a simulação corre durante vários minutos e uma falha pontual não deve interromper o fluxo completo.

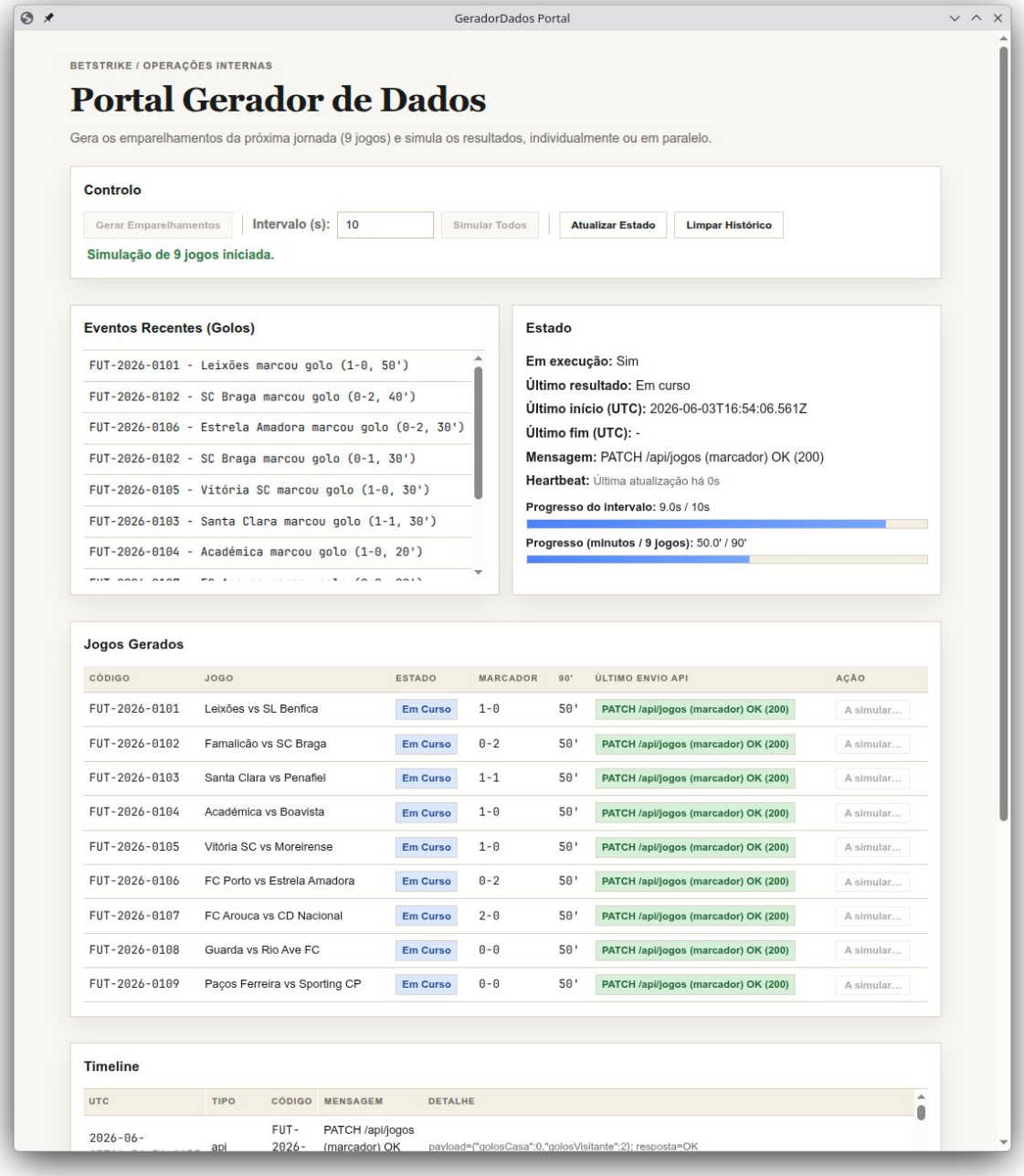


Figura 1 - Portal Gerador de Dados, durante uma simulação de 9 jogos em simultâneo

Através da aplicação geradora de dados, é ainda possível simular cada jogo de forma individual, havendo também a opção de ajustar o intervalo em segundos correspondente a dez minutos de jogo, por forma a agilizar as simulações.

3.3. Gestão de Estado dos Jogos

O modelo de estados (Agendado, EmCurso, Finalizado, Cancelado, Adiado) com transições estritamente controladas é um dos aspetos mais críticos do sistema. A validação existe em dois níveis: no controlador da API, que rejeita transições inválidas com HTTP 409 antes de tocar na base de dados; e nas Stored Procedures, que verificam novamente dentro da transação, protegendo contra race conditions. Esta dupla validação é intencional pois a camada de API oferece feedback rápido ao cliente, enquanto a SP garante consistência mesmo com acessos concorrentes.

Quando um jogo transita para Finalizado sem marcador associado, o sistema insere automaticamente um resultado 0-0 marcado como desconhecido. Esta decisão garante que toda a lógica de resolução de apostas funciona sem tratar casos especiais de resultado nulo, simplificando o código e eliminando uma classe inteira de bugs.

3.4. Integração ApiApostas e ApiPagamentos

A integração entre os sistemas de apostas e de pagamentos levantou um problema de atomicidade distribuída: criar um utilizador requer simultaneamente inserir um registo na base Apostas e criar um saldo inicial de 50,00 euros na base Pagamentos. Não é possível usar uma transação SQL única porque são bases de dados separadas.

A solução implementada é uma compensação síncrona: cria o utilizador, chama a ApiPagamentos, e se esta falhar, elimina o utilizador. Esta abordagem é simples e funcional para o volume em causa, mas tem uma janela de falha: se o processo crashar entre a chamada à ApiPagamentos e o return, o utilizador fica criado sem saldo. A alternativa robusta seria o padrão Saga, uma sequência de transações locais com ações de compensação coordenadas por eventos. A complexidade adicional não se justificava para o âmbito deste trabalho, sendo uma evolução natural para produção.

Para o fluxo de pagamentos de apostas, foi adotado um padrão de outbox simplificado. O campo PagamentoPendente (BIT) na tabela Apostas funciona como sinalizador: é marcado a 1 na mesma transação SQL da resolução, e a 0 apenas após a chamada HTTP à ApiPagamentos ser confirmada. Se a chamada falhar, o campo permanece a 1 e o PendingBetsSweeper (introduzido na Parte 2) trata do retry. Esta abordagem desacopla a atomicidade SQL da fiabilidade da chamada HTTP externa.

3.5. Frontend Angular

O frontend é uma SPA desenvolvida em Angular 21. Embora não seja o foco do trabalho, a sua existência é relevante para validar que a API está bem desenhada para clientes reais. As rotas, os contratos de dados, os mecanismos de autenticação e a gestão de estado são todos visíveis no frontend e servem como teste de integração informal.

Funcionalidades implementadas

A aplicação está organizada em cinco features com lazy loading:

- Autenticação: login e registo de utilizador. O token JWT devolvido pela API é armazenado em memória e injetado automaticamente nos pedidos seguintes via HTTP interceptor. O estado de autenticação é gerido por um AuthService com Angular Signals.
- Lista de jogos: tabela de jogos da jornada com filtros por data e estado, consumindo GET /api/jogos. Inclui polling automático a cada 10 segundos para refletir atualizações de marcador durante jogos EmCurso. Notificações de golo surgem no topo da página quando o marcador muda entre polling cycles.
- Detalhe do jogo: vista completa com resultado, odds atuais, estatísticas (total apostado, apostas por tipo, margem da plataforma) e formulário de aposta. Usa forkJoin para carregar dados do jogo, resultado e estatísticas em paralelo.
- As minhas apostas: histórico com estado atual (Pendente, Ganha, Perdida, Anulada), prémio potencial e opção de cancelamento para apostas ainda anuláveis.
- Painel de administração: visível apenas para a role Admin. Permite gerir jogos manualmente e visualizar todos os utilizadores e as suas apostas.

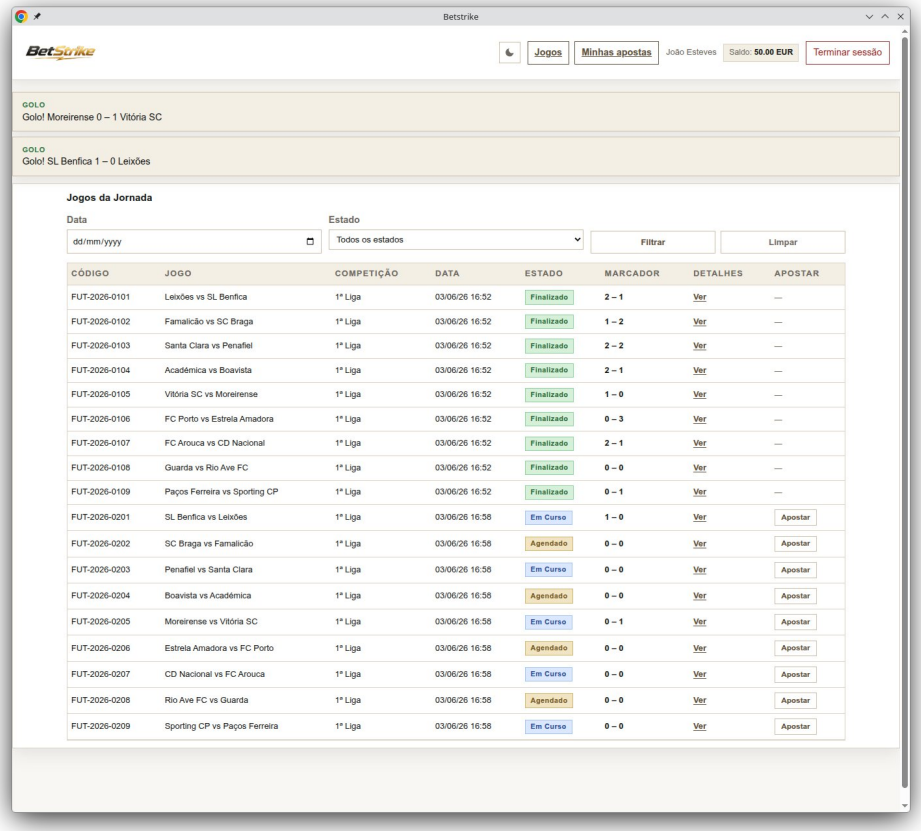


Figura 2 - Lista de jogos com filtros e notificações de golo (painel utilizador)

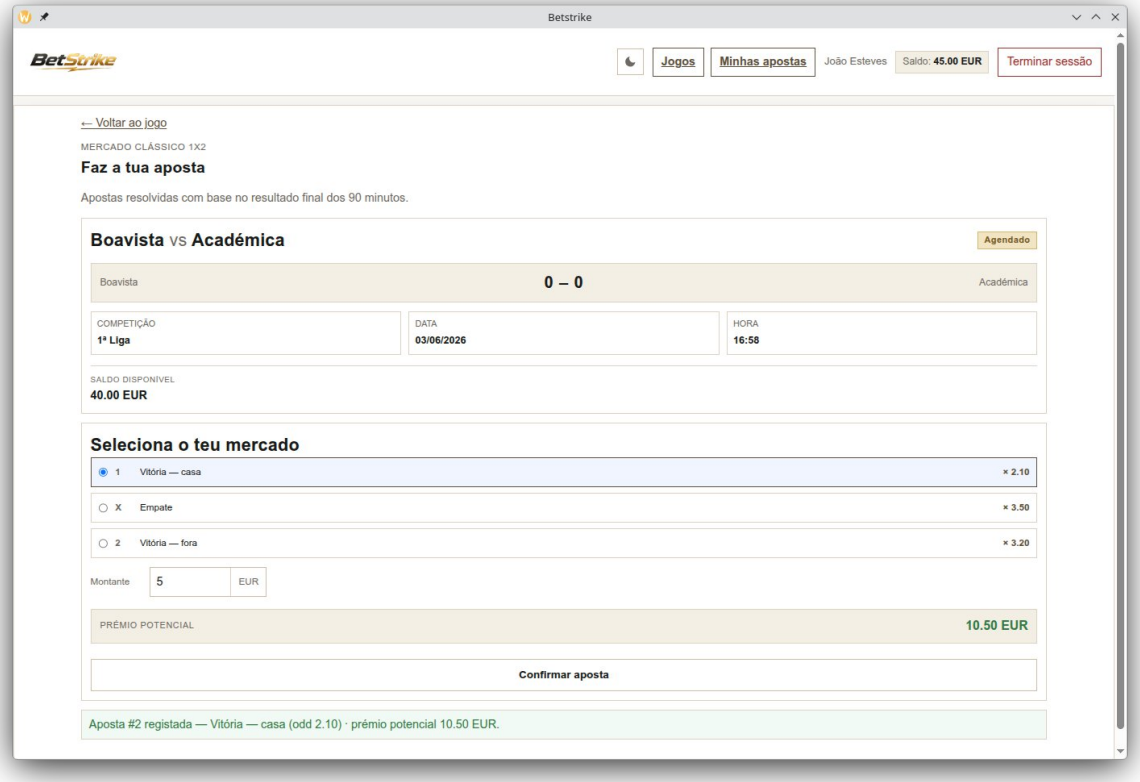


Figura 3 - Formulário de aposta (painel utilizador)

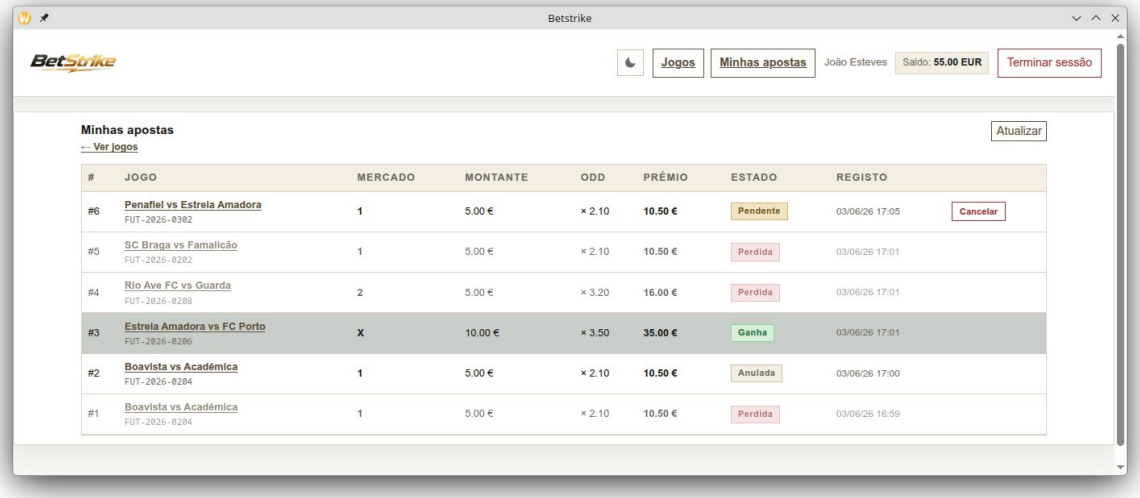


Figura 4 - Histórico de Apostas (painel utilizador)

Análise da relação com a API

O polling de 10 segundos na lista de jogos evidencia a limitação da Parte 1: sem eventos push, o cliente tem de perguntar periodicamente se algo mudou. Na Parte 2, a propagação via Kafka reduz a latência no backend, pelo que quando o cliente pergunta, os dados já estão atualizados. Uma evolução natural seria expor um endpoint SSE (Server-Sent Events) no ApiApostas alimentado pelos eventos Kafka, eliminando o polling do lado do cliente.

O ecrã de estatísticas chama os mesmos SPs que alimentam o ApiApostas. A separação entre as estatísticas transacionais (via Stored Procedure sp_EstatisticasJogo, consistentes e atómicas) e as métricas do AnalyticsService na Parte 2 (aproximadas mas instantâneas) é um reflexo da separação CQRS: o frontend pode escolher qual a fonte consoante o nível de consistência que precisa.

3.6. Containerização

A containerização com Docker Compose resolve o problema de reprodutibilidade do ambiente. O SQL Server, as APIs e o GeradorDados arrancam com um único comando, sem instalação manual de dependências. O db-init é um container one-shot que executa os scripts SQL de inicialização e termina. Os outros serviços aguardam a sua conclusão via depends_on: condition: service_completed_successfully antes de arrancar, garantindo que nunca tentam ligar à base de dados antes de o esquema estar criado.

A separação de credenciais num ficheiro .env não versionado e o uso de variáveis de ambiente para toda a configuração sensível garante que as imagens Docker são portáveis sem expor segredos no repositório.

4. Parte 2: Motivação para o Middleware

A Parte 1 resolveu a integração básica, mas deixou três problemas em aberto que se tornam críticos numa plataforma live:

- Acoplamento temporal: o ApiApostas dependia de polling HTTP periódico ao ApiResultados para sincronizar jogos. Se o ApiResultados estivesse em baixo, o ApiApostas ficava desatualizado. A propagação de um update levava até 10 segundos no pior caso.
- Processamento síncrono no caminho crítico: a resolução de apostas, que pode envolver dezenas de chamadas à ApiPagamentos, ocorria de forma síncrona na mesma chamada que atualizava o jogo, bloqueando o thread e aumentando o risco de timeout.
- Ausência de observabilidade: não havia forma de saber em tempo real quantas apostas estavam pendentes, qual a exposição financeira por jogo, ou se havia comportamentos anómalos.

A solução é Message-Oriented Middleware (MOM), infraestrutura que permite comunicação assíncrona entre serviços através de mensagens, desacoplando produtores de consumidores. Os produtores publicam mensagens sem saber quem as consome, e os consumidores processam ao seu ritmo sem depender da disponibilidade do produtor. Foram escolhidas duas tecnologias distintas porque os dois casos de uso têm semânticas fundamentalmente diferentes, como se analisa nas secções seguintes.

5. Apache Kafka: Streaming de Eventos

5.1. Fundamentos e Adequação ao Problema

O Apache Kafka é uma plataforma de streaming de eventos distribuída baseada no conceito de log distribuído imutável. As mensagens publicadas num tópico não são apagadas após o consumo, ficando retidas por um período configurável e identificadas por um offset numérico. Os consumidores avançam o seu offset de forma independente, o que tem duas consequências determinantes para este projeto:

- Fan-out nativo: o mesmo evento publicado pelo ApiResultados é consumido de forma completamente independente pelo ApiApostas e pelo AnalyticsService, em consumer groups distintos, sem qualquer coordenação entre eles e sem que o produtor precise de saber quantos consumidores existem. Adicionar o AnalyticsService não exigiu qualquer alteração no ApiResultados, o que ilustra o princípio Open/Closed ao nível da integração.
- Replay por offset: quando o AnalyticsService reinicia, reprocessa todo o histórico de eventos Kafka desde o offset 0 e reconstrói as suas métricas. Isto elimina a necessidade de snapshot inicial ou base de dados própria para o serviço de analytics.

5.2. Justificação da Escolha

Para a disseminação de eventos de jogo e aposta a múltiplos consumidores independentes, o Kafka é a escolha naturalmente adequada. O modelo de log distribuído imutável permite que o ApiResultados publique um único evento e que o ApiApostas e o AnalyticsService o consumam de forma completamente independente, sem coordenação entre eles e sem que o produtor precise de saber quantos consumidores existem. Esta propriedade de fan-out nativo seria possível com RabbitMQ através de um fanout exchange, mas com uma limitação determinante: as mensagens são apagadas após consumo, não existe replay. Se o AnalyticsService estivesse em baixo durante uma hora, perderia todos os eventos desse período sem possibilidade de recuperação. Com Kafka, o serviço reinicia e reprocessa o histórico completo a partir do offset 0.

Para a implantação foi utilizada a imagem Docker oficial em modo KRaft, por ser mais simples de operar em ambiente containerizado do que a configuração tradicional com ZooKeeper como componente separado.

5.3. Implementação e Decisões

O KafkaProducerService usa o padrão fire-and-forget (Produce() assíncrono com callback de erro), sem bloquear o thread do endpoint HTTP. Esta escolha prioriza a latência da operação transacional sobre a garantia de entrega ao Kafka. É uma troca consciente: em caso de downtime do Kafka, o evento perde-se. Para o caminho

financeiro crítico, esta limitação é compensada pelo PendingBetsSweeper. Para as métricas do AnalyticsService, o impacto é baixo, pois uma lacuna temporária é menos grave do que atrasar todas as respostas HTTP.

O commit de offsets é manual (EnableAutoCommit: false) no KafkaConsumerService: o offset só avança após o processamento completo da mensagem, incluindo a atualização da base de dados e, se aplicável, a publicação no RabbitMQ. Isto implementa at-least-once delivery. Em caso de falha antes do commit, o Kafka reentrega a mensagem no próximo arranque do consumer.

A chave de partição de cada mensagem é o CodigoJogo. Esta decisão garante que todos os eventos do mesmo jogo vão para a mesma partição, preservando a ordem de processamento: um evento game-updated com EmCurso nunca é processado antes do game-created correspondente.

5.4. Dois Tópicos, Dois Papéis

Tópico	Produtor	Consumidores	Eventos
game-events	ApiResultados	ApiApostas (apiapostas-jogos) AnalyticsService (analytics)	game-created, game-updated
bet-events	ApiApostas	AnalyticsService (analytics)	bet-placed, bet-cancelled, bet-resolved

O tópico bet-events permite ao AnalyticsService calcular métricas de apostas sem acesso à base de dados Apostas, respeitando o isolamento entre serviços. O campo competicao no evento bet-placed é obtido por consulta ao jogo antes de inserir a aposta, enriquecendo o evento com contexto que o AnalyticsService precisaria de ir buscar noutra chamada.

5.5. Análise do Fluxo Kafka

A substituição do polling HTTP pelo KafkaConsumerService mudou a arquitetura de pull para push. Na Parte 1, o ApiApostas perguntava periodicamente ao ApiResultados pelo estado mais recente, criando carga desnecessária e latência variável. Na Parte 2, o ApiResultados notifica proativamente todos os consumidores interessados no momento em que o estado muda, com latência inferior a 1 segundo na esmagadora maioria dos casos.

A limitação mais importante é o fire-and-forget no produtor. Se o Kafka estiver em baixo durante uma atualização de jogo, o evento perde-se e os consumidores nunca o recebem. O cenário 1 dos testes de resiliência confirmou este

comportamento. O Outbox Pattern resolveria esta limitação, mas foi excluído do âmbito por complexidade adicional.

6. RabbitMQ: Filas de Trabalho

6.1. Fundamentos e Adequação ao Problema

O RabbitMQ é um message broker baseado no protocolo AMQP, com semântica de work queue: cada mensagem é entregue a exatamente um consumidor, que confirma o processamento com ACK explícito. As mensagens são removidas da fila após o ACK. O modelo é diferente do Kafka: não se trata de disseminar eventos, mas de delegar trabalho a um worker que o processa ao seu ritmo.

A resolução de apostas é o caso de uso perfeito para uma work queue: quando um jogo termina, existe uma tarefa pesada e bem delimitada, resolver todas as apostas pendentes desse jogo, calcular prémios e creditar saldos. Esta tarefa não deve ser executada no thread do KafkaConsumerService (prolongaria o tempo de commit do offset), não deve ser duplicada (dois workers não podem resolver o mesmo jogo duas vezes), e deve ser retentada em caso de falha. O RabbitMQ satisfaz os três requisitos nativamente.

6.2. Justificação da Escolha

O Kafka poderia ser usado como work queue, bastaria ter um único consumer group com um único consumer. Mas a semântica é menos natural: o ACK no Kafka é um commit de offset que afeta todas as mensagens até esse ponto, e não há Dead Letter Queue nativa. O RabbitMQ tem ACK/NACK por mensagem individual, prefetchCount para backpressure controlado, e DLQ automática, tudo o que se precisa para um worker robusto com retry e isolamento de mensagens problemáticas.

Para além da adequação técnica, o RabbitMQ é uma tecnologia com a qual a equipa já estava familiarizada. Foi utilizado na unidade curricular de Sistemas Distribuídos na licenciatura, o que reduziu a curva de aprendizagem e permitiu focar o esforço na integração com os restantes componentes em vez de na aprendizagem da ferramenta.

6.3. Configuração e Estratégia de Retry

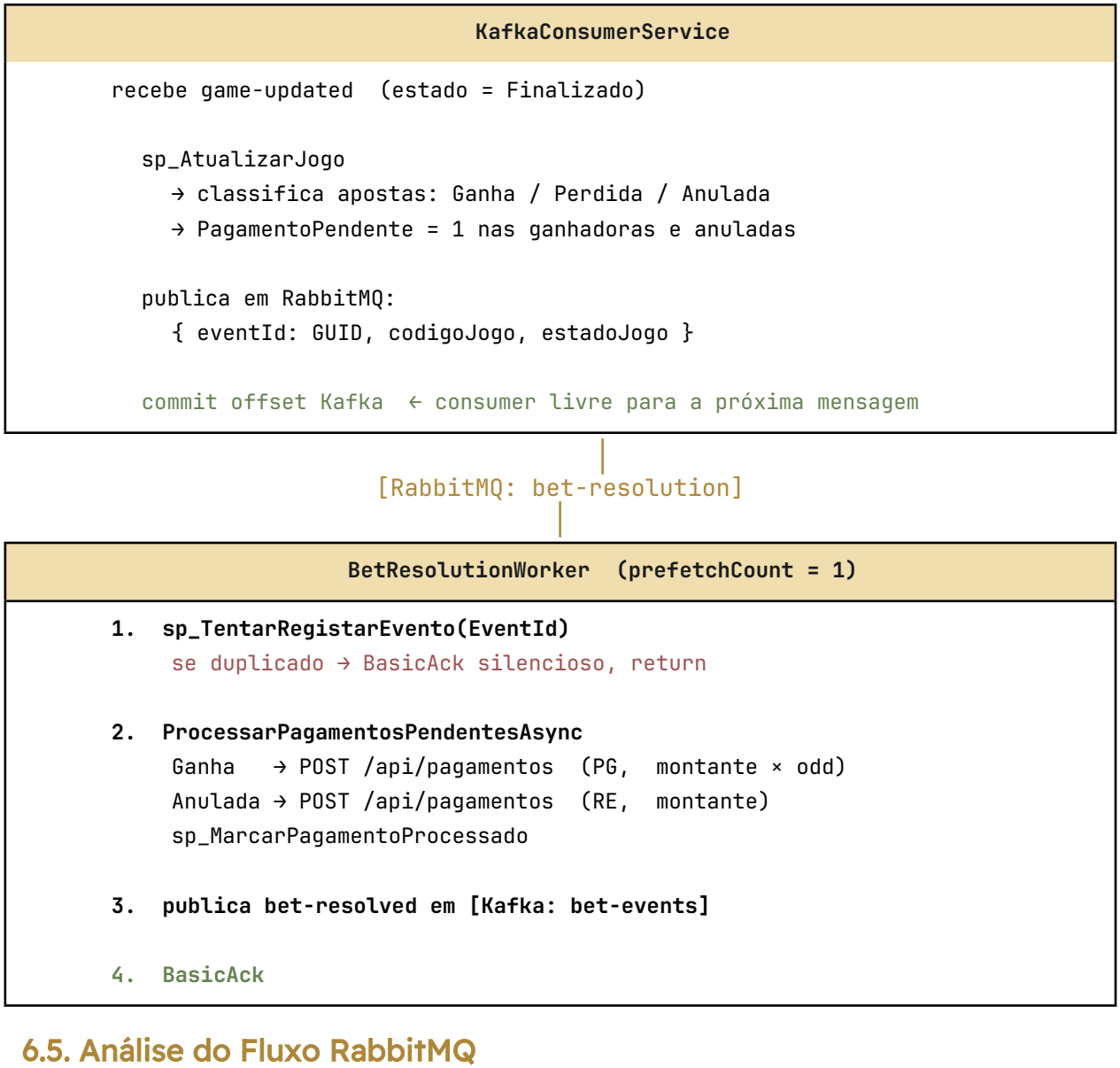
A fila bet-resolution é configurada como durable (sobrevive a restart do broker) com mensagens persistent (gravadas em disco). O TTL de 60 segundos garante que mensagens não processadas nesse tempo são movidas para a Dead Letter Queue, evitando que fiquem presas indefinidamente se o worker estiver em baixo.

O retry após falha usa um header próprio x-retry-count em vez de BasicNack(requeue:true). A razão é técnica: o RabbitMQ não permite alterar headers de uma mensagem existente. Para incrementar o contador de tentativas, é necessário publicar uma nova mensagem com o header atualizado e dar ACK à original. O header x-death, preenchido automaticamente pelo broker, só é populado após passagem pela

DLQ, tornando-o inutilizável no caminho normal de retry. Após 3 falhas, BasicNack(requeue:false) encaminha a mensagem para a DLQ via Dead Letter Exchange configurado na fila principal.

O prefetchCount igual a 1 implementa backpressure: o worker só aceita a mensagem seguinte depois de confirmar a atual. Isto evita que o worker acumule mensagens em memória se o processamento for lento, e garante que a fila reflete o trabalho real por processar, útil para monitorização via RabbitMQ Management UI.

6.4. BetResolutionWorker: Fluxo Completo



6.5. Análise do Fluxo RabbitMQ

A delegação da resolução de apostas para o BetResolutionWorker via RabbitMQ desacoplou completamente a atualização de jogos do processamento financeiro. O KafkaConsumerService trata do evento Kafka, publica na fila e faz

commit do offset em milissegundos, independentemente de quantas apostas precisam de ser resolvidas. O BetResolutionWorker processa uma mensagem de cada vez, sem pressão de latência.

O cenário de falha mais crítico, ApiPagamentos em baixo, demonstrou o comportamento do retry: 3 tentativas com header x-retry-count incrementado, seguidas de encaminhamento para a DLQ. A mensagem na DLQ pode ser inspecionada e re-injetada manualmente após resolução do problema. Este comportamento exige intervenção manual, o que é uma limitação real. Uma melhoria futura seria um Dead Letter Consumer com espera exponencial para re-injeção automática.

7. Idempotência e Resiliência

7.1. O Problema da Deduplicação

Com at-least-once delivery em ambas as camadas de middleware, a mesma mensagem pode chegar mais do que uma vez. Para o AnalyticsService isto é irrelevante. Para o BetResolutionWorker é crítico: processar duas vezes a resolução do mesmo jogo significaria creditar o mesmo prémio duas vezes.

A solução é o padrão Idempotency Key: cada mensagem bet-resolution inclui um EventId (GUID) gerado no momento da publicação. O worker verifica este identificador contra a tabela EventosProcessados antes de processar. A verificação usa INSERT com tratamento de violação de chave primária, e não SELECT seguido de INSERT, que teria uma janela de race condition entre as duas operações. A PK garante que dois threads concorrentes não conseguem ambos "ganhar" a inserção.

7.2. PendingBetsSweeper: Camada de Segurança

O PendingBetsSweeper é um BackgroundService com ciclo de 30 segundos que quebra a dependência entre a resolução de apostas e o estado momentâneo do RabbitMQ. Consulta todas as apostas com PagamentoPendente igual a 1, agrupa por jogo, e para jogos em estado terminal republica a mensagem de resolução com um novo EventId. A idempotência garante que, mesmo que o BetResolutionWorker já tenha processado o jogo entretanto, a mensagem duplicada é descartada sem efeitos.

Este padrão, sweeper periódico independente como safety net, é preferível a depender exclusivamente do retry automático do middleware porque é agnóstico relativamente ao estado do broker. Funciona mesmo que o RabbitMQ tenha estado em baixo por um período mais longo do que o TTL das mensagens.

7.3. Comportamento sob Falhas

Componente	Comportamento	Recovery
Kafka em baixo durante publicação	Evento perdido (fire-and-forget). ApiApostas não recebe atualização.	Imediato após restart. Limitação conhecida sem retry no produtor.
RabbitMQ em baixo durante publicação de bet-resolution	KafkaConsumerService não faz commit do offset. PendingBetsSweeper republica no ciclo seguinte (30s).	Automático. Saldos sempre corretos. Zero perda de apostas.
ApiPagamentos em baixo durante BetResolutionWorker	3 retries via x-retry-count. Após 3 falhas, mensagem vai para DLQ.	Requer re-injeção manual da DLQ via RabbitMQ UI.

Os resultados de resiliência confirmam que a combinação Kafka + RabbitMQ + Sweeper é suficiente para tolerar falhas temporárias de componentes sem perda de dados financeiros. O único cenário que requer intervenção manual é uma falha persistente do ApiPagamentos que exceda os 3 retries, o que implica um problema mais sério do que o próprio mecanismo de retry pode resolver autonomamente.

8. Analytics, Dashboards e Alertas

8.1. Abordagem Arquitetural

O AnalyticsService é o componente que resolve a ausência de visibilidade operacional. A decisão arquitetural central foi implementá-lo como microserviço independente que consome o stream Kafka, sem acesso direto às bases de dados transacionais. Esta separação implementa o padrão CQRS ao nível da infraestrutura: escritas ocorrem no caminho transacional, leituras analíticas consomem o stream de eventos derivado, sem interferência mútua.

As métricas são mantidas em estruturas de dados concorrentes em memória com janelas temporais deslizantes. A escolha de in-memory em vez de uma base de dados analítica foi deliberada: as métricas de curto prazo não justificam uma dependência adicional, e o replay Kafka elimina a necessidade de snapshot persistente para recuperação. O estado é persistido em JSON a cada 30 segundos como otimização de arranque, não como fonte de verdade.

8.2. Métricas e Alertas

O AnalyticsService calcula em contínuo: apostas por minuto e por 5 minutos, utilizadores ativos, taxas de vitória por tipo de aposta (1, X, 2), volume apostado por competição, exposição financeira por jogo (soma de montante vezes odd das apostas pendentes), volume total e margem da plataforma (volume menos prémios pagos). Todas as métricas são expostas via GET /api/analytics/resumo.

O sistema de alertas monitoriza quatro cenários anómalos com limiares configuráveis por variáveis de ambiente: exposição financeira elevada num jogo, cadência de apostas invulgar, e padrões anómalos de vitória ou derrota nos últimos 5 minutos. Os alertas são registados em log e acessíveis via API, satisfazendo o requisito de notificação automática do protocolo.

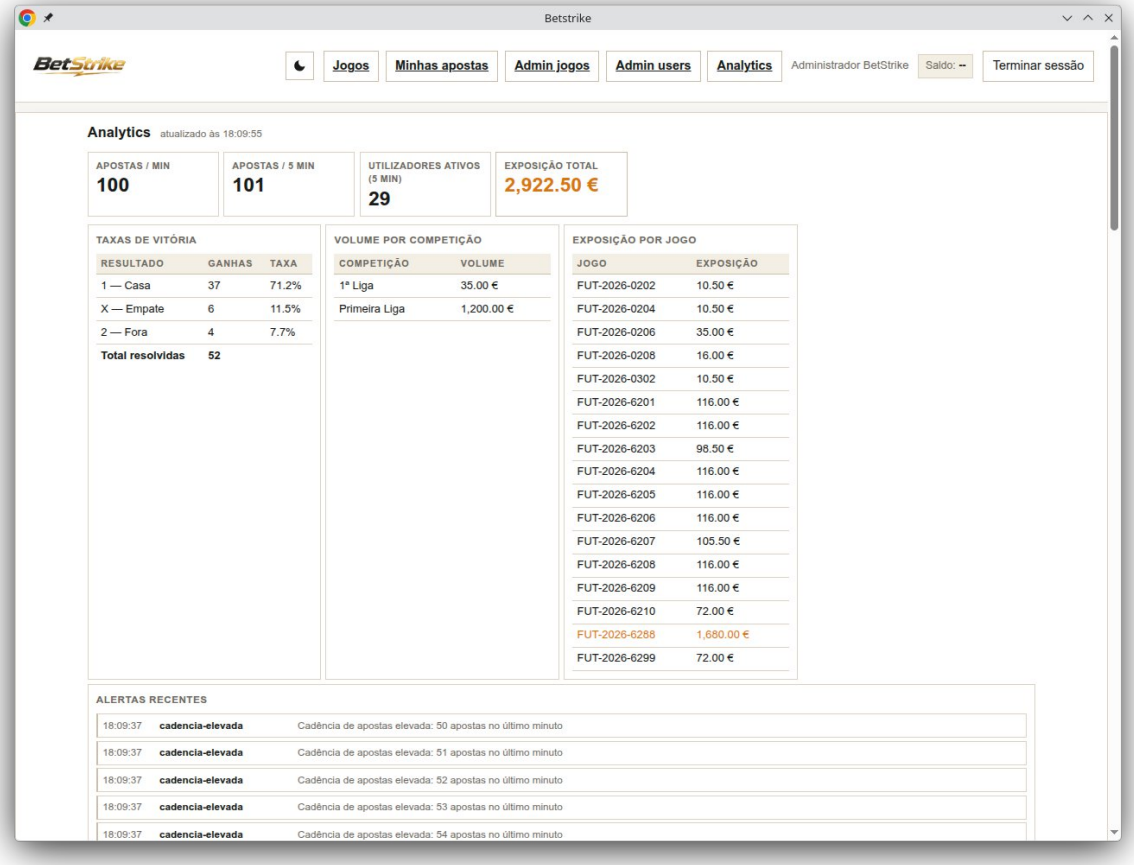


Figura 5 - Dashboard de analytics (administrador) com resumo das métricas e alertas em tempo real

8.3. Análise da Abordagem

A abordagem in-memory com replay Kafka é elegante mas tem uma limitação de escalabilidade: à medida que o volume de eventos históricos cresce, o tempo de reconstrução após restart aumenta. Para uma plataforma com meses de histórico, o replay completo poderia demorar minutos. A solução seria combinar um snapshot periódico com replay apenas dos eventos posteriores ao snapshot, ou usar uma base de dados de séries temporais como backend persistente.

Outro aspeto relevante é que as métricas do AnalyticsService são eventualmente consistentes: há uma latência de sub-segundo entre uma aposta ser colocada e a métrica de volume ser atualizada. Para o caso de uso (dashboards operacionais), esta consistência eventual é completamente aceitável. Seria problemática apenas para decisões transacionais, que nunca devem basear-se neste serviço.

9. Testes e Resultados

9.1. Testes de Integração

Uma coleção de 26 pedidos HTTP organizada em 7 pastas validou o fluxo completo contra o ambiente Docker: health checks de todos os serviços, registo e autenticação JWT, pipeline Kafka (publicação em ApiResultados e verificação de propagação em ApiApostas), colocação de apostas e verificação de métricas no AnalyticsService, resolução de apostas via RabbitMQ com verificação de crédito, cancelamento com reembolso, e estado final de analytics. Resultado: 26/26 passaram, 0 falhas, em cerca de 10 segundos.

9.2. Testes de Stress

Script	Jogos	Apostas	Falhas	Duração	Cenário
stress.mjs	5	40	0	~9s	Pipeline completo básico
stress2.mjs	12	100	5*	~11s	High-stakes, cancelamentos, alertas ativados
stress3.mjs	25	905	0	~13s	3 fluxos paralelos: normal, cancelamento, sem apostas
stress4.mjs	60	30 000	0	~130s	Throughput máximo: ~260 apostas/segundo

As 5 falhas de stress2 são rejeições de saldo esperadas: utilizadores que tentaram apostar em múltiplos jogos em simultâneo com saldo insuficiente. O sistema rejeitou corretamente sem debitar saldo. O bottleneck a 260 apostas/segundo é a sp_InserirAposta, que executa numa transação atómica com validação de saldo, verificação de estado do jogo e inserção. A atomicidade é um requisito funcional, não algo que se possa sacrificar por performance.

9.3. Testes de Resiliência e Idempotência

Os três cenários de falha (Kafka, RabbitMQ e ApiPagamentos em baixo) foram validados com o script resilience.mjs. O resultado mais relevante é o cenário de RabbitMQ em baixo: o PendingBetsSweeper ativou-se 30 segundos após o restart do

broker, republicou as mensagens pendentes, e os saldos dos utilizadores foram creditados corretamente sem perda de dados nem intervenção manual.

A tabela EventosProcessados após os testes completos continha 56 registos para 56 jogos distintos, confirmando que cada jogo foi resolvido exatamente uma vez apesar das reentregas geradas pelos cenários de falha.

10. Análise Crítica e Limitações

10.1. O que Funciona Bem

- Desacoplamento via Kafka: adicionar o AnalyticsService não exigiu qualquer alteração em ApiResultados ou ApiApostas. O sistema está genuinamente aberto à extensão.
- Resiliência da resolução de apostas: a combinação Kafka, RabbitMQ e Sweeper garante que nenhuma aposta fica por resolver, mesmo com falhas de componentes.
- Idempotência por EventId: simples, portável e sem race conditions. A tabela EventosProcessados é auditável, sendo possível ver exatamente o que foi processado e quando.
- Separação transacional e analítica: o AnalyticsService nunca concorre com as bases de dados transacionais. Picos de carga no analytics não afetam as apostas.

10.2. Limitações e Melhorias Futuras

- Fire-and-forget no Kafka: se o broker estiver em baixo durante uma atualização de jogo, o evento perde-se. O Outbox Pattern resolveria isto, com um worker de relay para o Kafka a partir de uma tabela SQL. Escolhemos não implementar pela complexidade adicional, aceitando a limitação conscientemente.
- DLQ requer intervenção manual: após 3 falhas de ApiPagamentos, a re-injeção é manual via RabbitMQ UI. Um Dead Letter Consumer com espera exponencial automatizaria este processo.
- Analytics in-memory sem alta disponibilidade: um restart com perda do volume Docker obriga a replay completo do Kafka. Para produção seria preferível uma base de dados de séries temporais (InfluxDB, TimescaleDB) como backend persistente.

11. Conclusão

O trabalho implementou com sucesso a modernização da plataforma BetStrike em duas fases progressivas. A Parte 1 eliminou os processos manuais e estabeleceu a integração automática entre os sistemas, com robustez garantida por Stored Procedures transacionais e containerização Docker. A Parte 2 introduziu middleware de mensagens que transformou a arquitetura: o Kafka eliminou o acoplamento síncrono e habilitou fan-out de eventos com replay; o RabbitMQ delegou a resolução de apostas para processamento assíncrono com retry e Dead Letter Queue; o AnalyticsService forneceu a visibilidade operacional que estava em falta.

A decisão de usar dois middlewares distintos em vez de um só reflete a compreensão da diferença fundamental entre streaming de eventos (Kafka) e delegação de trabalho (RabbitMQ). Cada tecnologia foi aplicada ao paradigma onde a sua semântica é naturalmente adequada, resultando numa solução mais clara e mais robusta do que uma solução homogénea teria sido.

Os testes de stress com 30 000 apostas a cerca de 260 por segundo e os testes de resiliência com três cenários de falha confirmaram que o sistema se comporta corretamente sob carga e em condições adversas. As limitações identificadas são tradeoffs conscientes, documentados, com caminhos de evolução conhecidos.

12. Referências Bibliográficas

- Hohpe, G., & Woolf, B. (2004). Enterprise Integration Patterns: Designing, Building, and Deploying Messaging Solutions. Addison-Wesley Professional.
- Erl, T. (2004). Service-Oriented Architecture: A Field Guide to Integrating XML and Web Services. Prentice Hall.
- Oberle, D. (2006). Semantic Management of Middleware (Vol. 1). Springer Science & Business Media.
- Curry, E., Scerri, S., & Tuikka, T. (2022). Data Spaces: Design, Deployment and Future Directions. Springer Nature.
- Surianarayanan, C., Ganapathy, G., & Pethuru, R. (2019). Essentials of Microservices Architecture: Paradigms, Applications, and Techniques. Taylor & Francis.
- Madni, A. M., & Sievers, M. (2014). Systems Integration: Key Perspectives, Experiences, and Challenges. Systems Engineering, 17(1), 37-51.
- Confluent, Inc. (2024). Apache Kafka Documentation.
- Broadcom Inc. (2024). RabbitMQ Documentation: AMQP 0-9-1 Protocol, Dead Letter Exchanges.
- Microsoft. (2024). ASP.NET Core: Background Services e Health Checks. Microsoft Learn.

Anexo 1 - Código Fonte

O código fonte deste projeto está disponível publicamente no GitHub em https://github.com/joaoesteves10/IS-BetStrike